Formula 110 — Exploration of Two Experimental Approaches

Per the assignment's exploration stage, we investigated two approaches that differ in mechanism, not just parameters: **reactive control with parameter optimization** (hand-written rules, tuned) and **neuroevolution** (a learned policy, evolved). Both are evaluated with the same headless harness (`scripts/evaluate_controller.py`) and the same fitness definition — scored distance, heavily penalized on elimination — so results are comparable.

Approach 1: Reactive Control + Parameter Optimization

Hypothesis: The simulator's processed sensors (`camera.heading_error_degrees`, `camera.center_offset_m`, `wall_lidar`) already encode steering error and wall proximity directly. A small set of explicit rules connecting these to throttle/steer should produce a controller that survives and completes laps almost immediately, and tuning its parameters (by hand or by search) should recover most of the achievable speed without needing training infrastructure.

Minimum experiment: A parametrized rule-based controller (`ReactiveParams`: steering gains for centerline/heading/lookahead + wall-avoidance, a speed-scaled braking distance against `wall_lidar.front_m`) evaluated headless, then manually adjusted once based on measured slack (zero damage) rather than guesswork.

Evaluation: Seed suite `(42, 110, 271, 997, 2027)`, 5 races per seed (25 total), 30-second rounds. Metrics: survival rate, lap-completion rate, scored distance, damage, max speed.

Status: implemented, redesigned, and parameter-optimized.

The controller went through two stages of improvement past the initial hand-tuned baseline:

1. **Redesign** (apex-hugging steering + no proactive cornering brake): steering now biases toward the inside of an upcoming turn — inferred from `heading_error_degrees` — rather than the raw centerline, since the simulator scores centerline *projection*, so cutting a turn's inside advances scored distance per meter driven. Throttle always targets top speed; the only source of braking is the reactive wall-proximity check.
2. **Parameter search** (`scripts/optimize_reactive.py`): a (mu + lambda) evolution strategy seeded from the redesigned defaults, with a diagnostic tracer that reports per-tick sensor/command state to explain *why* a parameter set does well or poorly (e.g. it revealed the pre-search defaults were braking on 43% of ticks — not from cornering logic, but because the safety margin scaled too conservatively with speed).

Stage	Survival	Laps	Damage	Avg. scored distance	Max speed
Original hand-tuned baseline	25/25	25/25 (≥1)	0.00	~229 m	13.9 m/s
+ Apex-hugging redesign (no search yet)	25/25	25/25 (≥1)	0.00	~259 m (+13%)	13.9 m/s

Stage	Survival	Laps	Damage	Avg. scored distance	Max speed
+ Parameter search	25/25	25/25 (2 laps)	0.00	**~435 m (+90%)**	15.7 m/s

Metric	Result
Development effort	Low-to-moderate — one redesign pass, one ~3-minute search run
Remaining risk	`steer_limit` settled at 0.63 (well under the 1.0 max), suggesting there may be more speed available in the sharpest corners

Files: `src/controllers/reactive.py`, `scripts/evaluate_controller.py`, `scripts/optimize_reactive.py`. Full experimental log: `LAB_NOTEBOOK.md`, Entries 1 and 3.

Approach 2: Neuroevolution (Evolution Strategy over a Small MLP)

Hypothesis: A small neural network (10 normalized sensor inputs → 6-unit hidden layer → 2 outputs, ~80 weights) evolved directly against race distance can discover throttle/steer behavior without hand-specifying rules, and — because it optimizes the actual scoring objective end-to-end — may find a faster or more consistent policy than hand-written rules, at the cost of needing many simulated races to train.

Minimum experiment: A dependency-free (mu + lambda) evolution strategy: a population of random-weight genomes, evaluated by scored distance with a penalty on elimination, with elitism and Gaussian mutation refilling each next generation. Kept intentionally simple — no covariance adaptation, no external library — to bound development effort for a first pass.

Evaluation: Trained on seeds distinct from the recommended suite (fast, 20-second rounds); validated on the held-out suite `(42, 110, 271, 997, 2027)`, 5 races per seed, 30-second rounds — identical protocol to Approach 1 for a direct comparison.

Status: implemented and trained; now reliable. Five training runs were needed to understand the fitness landscape, each revealing a distinct failure mode:

Attempt	Fitness design	Training result	Held-out validation (25 races)
1	Distance − 300 if eliminated	Converged to 225 m	8/25 survived, 18/25 laps — **fast but reckless**: crashing far enough to still out-score cautious genomes
2	Distance replaced by fixed −50 if eliminated (survival-only)	Converged to 63 m	20/25 survived, **0/25 laps**, ~0 m avg — discovered that standing still is a free, zero-risk "safe" strategy

Attempt	Fitness design	Training result	Held-out validation (25 races)
3	Distance − 350 if eliminated, − 20 if idle (< 10 m)	Converged to 260 m (single training seed)	8/25 survived, 21/25 laps — fast (28–35 m/s) and usually completes a lap, but still crashes in most races
4	Same fitness as #3, trained on two seeds instead of one	Converged to 53 m	Reproduced attempt #2's idle genome exactly — the wider requirement (safe on two starting points) overwhelmed this population/generation budget
5	Same fitness as #3/#4, trained on **five** seeds, with population/generations scaled up (10→20, 6→15) to match the harder objective	First tried at the old population/generations budget: converged to only 8.8 m — confirmed seed count alone made the problem harder without more search capacity to solve it. Rerun at population 20 / generations 15: converged to 235.8 m	**24/25 survived, 25/25 laps**, avg scored distance **356.0 m**, max speed 28.9 m/s — the first neuroevolution genome that generalizes across the held-out suite instead of overfitting to one or two training starts

Attempt 5's genome is now `BEST_GENOME` in `src/controllers/neuro.py`, replacing attempt 3's — it is both the fastest-converging *and* the most reliable neuroevolution result so far.

Metric	Result (Attempt 5, held-out suite)
Survival rate	24/25 races
Lap-completion rate	25/25 races
Avg. scored distance	356.0 m
Max speed reached	28.9 m/s (~1.8× the reactive controller's 15.7 m/s)
Development effort	High — five training iterations total, plus a within-attempt lesson that seed count and search budget must scale together
Remaining risk	One held-out race still ended in elimination (damage 1.00, seed 997) — not yet a perfect safety record; fitness still uses the mean across training seeds, so an outlier-punishing (e.g. worst-case or spread-penalized) objective is untried

Files: `src/controllers/neuro.py`, `scripts/train_neuroevolution.py`.

Comparison Summary

	Approach 1: Reactive (optimized)	Approach 2: Neuroevolution
Survival rate (held-out suite)	25/25	24/25
Lap-completion rate	25/25 (2 laps each)	25/25 (1–2 laps)
Avg. scored distance	~435 m	356.0 m
Max speed	15.7 m/s	28.9 m/s
Development effort	Low-to-moderate — one redesign pass, one ~3-minute search run	High — five training iterations to diagnose reward shaping and search-budget scaling
Interpretability	High (readable rules)	Low (opaque weights)
Remaining risk	`steer_limit` below max suggests unclaimed speed in sharp corners	One held-out elimination remains; fitness still averages across seeds rather than penalizing the worst one

Reading the evidence: Approach 1 still leads on survival, lap completion, and average distance, but the gap has closed substantially since attempt 5 — Approach 2 now survives 24/25 held-out races (up from 8/25) and completes a lap in all 25, while still holding a ~1.8× top-speed advantage (28.9 m/s vs. 15.7 m/s). The decisive lesson from attempt 5 was that Approach 2's earlier failures were as much a *search-budget* problem as a *fitness-design* problem: requiring generalization across 5 seeds only worked once population and generations were scaled up to match the harder objective — at the old budget, more seeds made results strictly worse (235.8 m final fitness vs. 8.8 m). Both approaches also share an older lesson — seed a search from a known-good baseline rather than random initialization — which is likely why Approach 1's search has never hit a degenerate optimum the way Approach 2's random-init runs originally did. Approach 1 remains the safer default given its perfect record, but Approach 2 is no longer clearly worse overall — it is a live candidate, and the remaining gap (one elimination, lower average distance) is a narrower, more specific problem than it was before attempt 5.